POC 2 - Runtime

Appel d'offres ARES · C++ (ou C)

Contexte

Le runtime exécute les bots et les traite comme hostiles : il les lance dans des process séparés, lit ce qu'ils produisent, et les arrête s'ils ne donnent aucun signe de vie à temps. Le reste de la plateforme ne touche jamais un bot directement, il passe par le runtime.

Fourni

- Le container C++ pour DevContainer
- Trois bots de test, qui servent de cibles :

Bot	Comportement	Verdict attendu
`bot_ok`	Répond immédiatement	`OK`
`bot_slow`	Attend 5 s puis répond	`TIMEOUT`
`bot_loop`	Boucle sans jamais lire son entrée	`TIMEOUT`

À faire

Écrire un programme qui lance un bot dans un process séparé (`fork` + `execvp`) et **lit sa sortie**. Si le bot imprime quelque chose dans les **500 ms**, il est vivant : verdict `OK` . Si rien n'arrive dans ce délai, le programme l'arrête : verdict `TIMEOUT` .

Exécution attendue :

```
$ ./runtime ./bot_ok
OK      (12 ms)
$ ./runtime ./bot_slow
TIMEOUT (500 ms, killed)
$ ./runtime ./bot_loop
TIMEOUT (500 ms, killed)
$ ps -C bot_loop
  (aucun process)
```

Contrainte

Après un `TIMEOUT` , aucun process enfant ne doit survivre. La preuve en est fournie dans le README, sous la forme d'une sortie `ps` après l'exécution des trois cas.

Pour démarrer

Il s'agit sans doute de votre premier vrai programme système. L'annexe « Exécuter un code hostile » présente les appels nécessaires (`fork` , `exec` , `pipe` , `poll` , `kill` , `waitpid`), leur rôle et un ordre de progression.

Rendu

- Le code du lanceur.
- Un README court : la sortie `ps` qui prouve qu'aucun process ne subsiste, et les difficultés rencontrées.